

Lecture 05 | July, 2026

LLM as a Judge

Instructor: Sithvothy Kiv

Slides Credit: Josh Hug

LLM as a Judge

- **LLM as a judge**
- Loading the RAG answers
- A->Q->A' evaluation
- Running the judge
- Evaluating the judge
- Saving the results

LLM as a Judge

In the previous lesson, we generated RAG answers for our ground truth questions. Now we need to decide whether these answers are good enough.

For offline evaluation, we have three things:

- the original FAQ answer
- the question generated from that answer
- the answer generated by our RAG pipeline

An LLM judge is another LLM call that compares these three pieces.

We ask it whether the RAG answer recovers the same information as the original answer.

LLM as a Judge

It can also explain why an answer is bad.

For example:

- the retrieved document might be wrong
- the answer might miss the key point
- the model might say that it doesn't know

LLM as a Judge

This approach is useful when exact text matching is too strict. The RAG answer doesn't need to copy the FAQ answer word for word.

It needs to answer the question with the same key information.

This evaluates the full RAG flow in one pass:

- search: did we retrieve context that contains the answer?
- prompt: did we give the model enough context to answer?
- LLM: did the model produce a useful answer from that context?

If the judge marks an answer as bad, we still need to look at the example.

The judge tells us where to investigate. It doesn't replace reading the failing cases.

Loading the RAG answers

- LLM as a judge
- **Loading the RAG answers**
- A->Q->A' evaluation
- Running the judge
- Evaluating the judge
- Saving the results

Loading the RAG answers

Start from the CSV we created in the previous lesson:

```
import pandas as pd

df_answers = pd.read_csv("data/rag-answers-new.csv")
answers = df_answers.to_dict(orient="records")
```

Each row has the generated question, the original FAQ answer, and the answer produced by the RAG pipeline.

This is offline evaluation. We can do it because our test dataset came from FAQ records.

We know the original answer for every generated question.

Loading the RAG answers

In production, we usually don't have that original answer for real user questions. There we can still use an LLM judge. The prompt has to judge only the question and the generated answer.

In this lesson, we use the stronger offline setup.

A->Q->A' evaluation

- LLM as a judge
- Loading the RAG answers
- **A->Q->A' evaluation**
- Running the judge
- Evaluating the judge
- Saving the results

A->Q->A' evaluation

We'll compare the RAG answer with the original answer from the FAQ. This checks if the RAG pipeline is producing answers that match the ground truth.

First, define the output format:

```
from pydantic import BaseModel, Field
from typing import Literal

class AnswerEvaluation(BaseModel):
    reasoning: str = Field(
        description="Reasoning about the quality of the answer."
    )
    score: Literal["good", "bad"] = Field(
        description="good' if the answer is correct and complete, 'bad' otherwise."
    )
```

A->Q->A' evaluation

The judge returns two fields. The score gives us a metric we can aggregate. The reasoning explains the score, which helps when we look at bad examples.

A->Q->A' evaluation

First, write the judge instructions. This tells the judge what to compare and how to assign the score.

```
aqa_judge_instructions = ""
```

You are an expert evaluator. You will be given:

1. A question from a student
2. The original answer from the FAQ (ground truth)
3. An answer generated by an AI assistant

Your task is to decide if the AI answer is semantically equivalent to the original answer.

Rules:

- The AI answer does NOT need to be word-for-word identical
- It should convey the same key information
- Extra detail is fine as long as the core answer is correct
- Mark 'bad' only if the AI answer is wrong or misses the key point

Be fair and focus on correctness, not style.

```
"".strip()
```

A->Q->A' evaluation

Then define the prompt template. This is the data we pass to the judge for each answer.

```
aqc_judge_prompt = """
```

```
Question:
```

```
{question}
```

```
Original Answer (ground truth):
```

```
{answer_orig}
```

```
AI Answer:
```

```
{answer_llm}
```

```
""".strip()
```

A->Q->A' evaluation

Import the structured-output helper:

```
from openai import OpenAI
from evaluation_utils import calc_price, calc_total_price, llm_structured_retry, map_progress

openai_client = OpenAI()

# Take one record:
rec = answers[0]
```

A->Q->A' evaluation

Create the judge prompt:

```
prompt = aqa_judge_prompt.format(  
    question=rec["question"],  
    answer_orig=rec["answer_orig"],  
    answer_llm=rec["answer_llm"]  
)  
  
# Call the judge:  
eval_result, usage = llm_structured_retry(  
    openai_client,  
    aqa_judge_instructions,  
    prompt,  
    AnswerEvaluation,  
)  
  
eval_result
```

A->Q->A' evaluation

Now put the same logic into a function:

```
def evaluate_aqa(question, answer_orig, answer_llm, model="gpt-5.4-mini"):
    prompt = aqa_judge_prompt.format(
        question=question,
        answer_orig=answer_orig,
        answer_llm=answer_llm
    )

    result, usage = llm_structured_retry(
        openai_client,
        aqa_judge_instructions,
        prompt,
        AnswerEvaluation,
        model=model,
    )

    return result, usage
```

A->Q->A' evaluation

Test it on the same record:

```
eval_result, usage = evaluate_aqa(  
    question=rec["question"],  
    answer_orig=rec["answer_orig"],  
    answer_llm=rec["answer_llm"]  
)  
  
eval_result
```

Running the judge

- LLM as a judge
- Loading the RAG answers
- A->Q->A' evaluation
- **Running the judge**
- Evaluating the judge
- Saving the results

Running the judge

Run the evaluation on all answers:

```
def judge_record(rec):
    eval_result, usage = evaluate_aqa(
        question=rec["question"],
        answer_orig=rec["answer_orig"],
        answer_llm=rec["answer_llm"]
    )

    result = {
        "question": rec["question"],
        "document": rec["document"],
        "score": eval_result.score,
        "reasoning": eval_result.reasoning,
    }

    return result, usage
```

Running the judge

Use the same parallel processing helper:

```
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=6) as pool:
    results = map_progress(pool, answers, judge_record)
# Split the results:

evaluations = []
usages = []

for evaluation, usage in results:
    evaluations.append(evaluation)
    usages.append(usage)
```

Running the judge

Create a dataframe:

```
df_eval = pd.DataFrame(evaluations)
# Calculate the total cost:

calc_total_price(usages)

# Check the results:

good_count = (df_eval["score"] == "good").sum()
total_count = len(df_eval)
print(f"Good: {good_count}/{total_count} = {good_count/total_count:.2%}")
```

Running the judge

Look at the "bad" cases to understand what went wrong:

```
df_eval[df_eval["score"] == "bad"].head()
```

These rows are often the most useful part of the evaluation. They can show that search retrieved the wrong document. They can also show that the answer is too generic.

Sometimes the RAG pipeline says that it doesn't know even though the FAQ had the answer.

Evaluating the judge

- LLM as a judge
- Loading the RAG answers
- A->Q->A' evaluation
- Running the judge
- **Evaluating the judge**
- Saving the results

Evaluating the judge

The judge can be wrong. It may rate an answer as good even though search failed to retrieve the right document. In that case the judge is too lenient. Make the instructions stricter and re-run the evaluation.

To evaluate the judge, you need to look at the results yourself. Sample some good and bad cases, read the judge reasoning, and check whether you agree with the verdict.

You cannot use another judge to evaluate the judge. This is manual work, but it is necessary.

Evaluating the judge

A practical approach is to build a simple application with Streamlit. Show each question, the original answer, the generated answer, and the judge verdict side by side.

Then mark each verdict as correct or incorrect and use that feedback to adjust the judge instructions. This is a lot of trial and error, but it makes the evaluation framework more reliable.

Saving the results

- LLM as a judge
- Loading the RAG answers
- A->Q->A' evaluation
- Running the judge
- Evaluating the judge
- **Saving the results**

Saving the results

Save the judged answers:

```
df_eval.to_csv("data/rag-evaluations-new.csv", index=False)
```

We now have an answer-quality score for the RAG pipeline. In the next lesson, we'll apply the same idea to an agent and also capture the tool calls it made.